

# מבוא למדעי המחשב

שיעור 0: מבוא למבוא

לקריאה נוספת:

- B. J. Copeland. *The modern history of computing*. Stanford Encyclopedia of Philosophy, 2006.
- R. X. Cringely. *Triumph of the Nerds*. PBS, 1996.
- D. E. Knuth. *Ancient Babylonian Algorithms*. *CACM* 15(7): 671-677, July 1972.

# אודות הקורס

- הצוות: יובל רבני (המרצה), תאיר שאולי (המתרגלת), זיו בן אהרון (הצאר), בודקי תרגילים.
- באתר הקורס ב-moodle אפשר למצוא מידע על שעות הקבלה, תרגילי הבית, הבחינה, פורום לשאלות על התרגילים, וכו':  
<https://moodle2.cs.huji.ac.il/nu17/course/view.php?id=6710102>
- אין ספר מומלץ.
- הרכב הציון: 50% תרגילי הבית (~מידי שבוע), 50% בחינה מסכמת.
- יש לקבל ציון עובר (60) הן בממוצע התרגילים והן בבחינה.
- אסור בתכלית להעתיק, הן בתרגילים והן בבחינה.
- **אסור**: להתבונן בקוד, להעתיק קבצים, לקבל הנחיות כתיבה  
**מותר**: להחליף דעות על התרגיל (בע"פ, לא מול קוד)

# מה נלמד?

- תכנות בשפת פייתון
- עקרונות של תכנון ומימוש מערכות תוכנה
- חשיבה אלגוריתמית
- טעימה ממגוון נושאים במדעי המחשב

קצת רקע

# חשוב הוא תופעת טבע



# קצת הסטוריה ...



Gottfried Wilhelm Leibniz 1646-1718

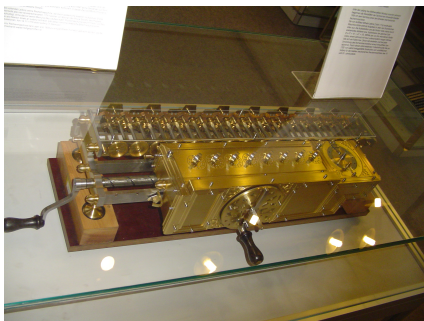
חזון: מיכון החשיבה הלוגית

שני מרכיבים:

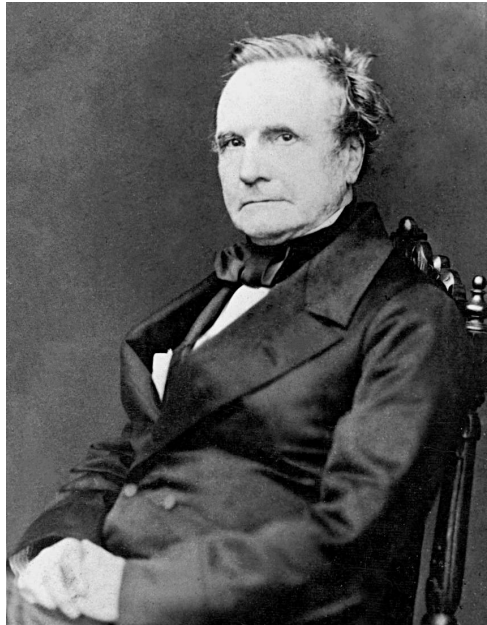
- שפה פורמלית אוניברסלית *Characteristica Universalis*
- תחשיב לוגי *Calculus Ratiocinator*

מכונת החישוב (*Staffelwalze*) של לייבניץ

פעולות חשבון: חיבור, חיסור, כפל, חילוק



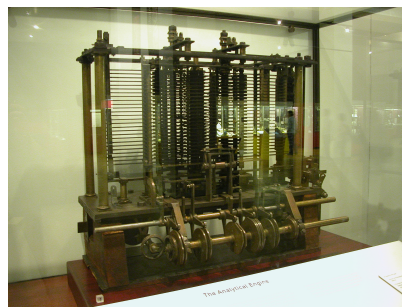
# חישוב מכני



Charles Babbage 1791-1871



1822 מנוע ההפרשים  
חישובים נומריים: טורי טיילור



1834 המנוע האנליטי  
חישוב כללי

# הגדרה מתמטית של חישוב

1920 התוכנית של הילברט David Hilbert

להוכיח שהמתמטיקה היא עקבית, שלמה וכריעה

1931 הקידוד של גדל Kurt Gödel

משפטי אי-השלמות

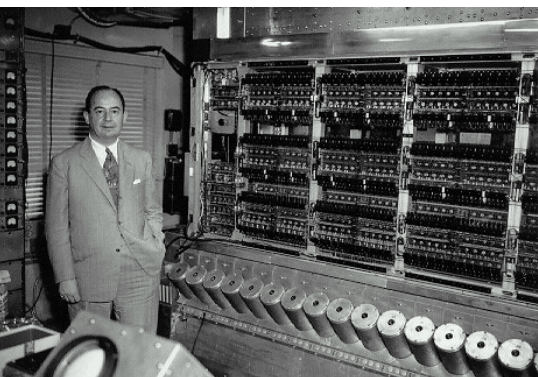
אם תורת המספרים עקבית, אז יש משפט שלא ניתן להוכיחו או להפריכו  
(נדרשת "שיטה אפקטיבית" לבדוק הוכחות - מערכת לוגית אפקטיבית)

1936 מכונת טיורינג Alan Turing

קיימת מכונה אוניברסלית

אי אפשר להכריע את בעיית העצירה (שניתנת לניסוח בתורת המספרים)

תחשיב λ Alonzo Church



# מחשב אוניברסלי

George Boole

1848 אלגברה בוליאנית

Claude Shannon

1937 מעגלים דיגיטליים

ניתן לממש אלגברה בוליאנית ע"י מערכים של ממסרי טלפוניה אלקטרו-מכניים

John von Neumann

1945 EDVAC

הארכיטקטורה של פון-נוימן (stored-program computer):  
התוכנית מאוכסנת בזיכרון כמו הנתונים.

Claude Shannon

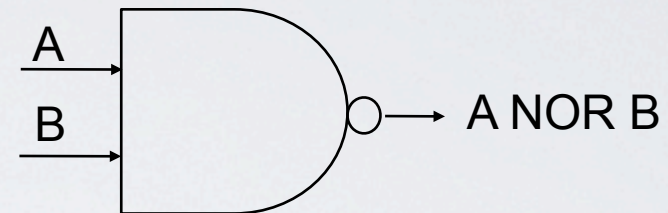
1948 תורת האינפורמציה

קודים בינאריים לתיקון שגיאות.

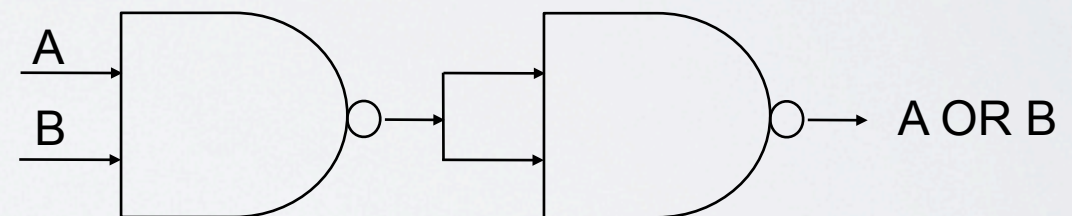
חומרה

# שער NOR

| A | B | A NOR B |
|---|---|---------|
| 0 | 0 | 1       |
| 0 | 1 | 0       |
| 1 | 0 | 0       |
| 1 | 1 | 0       |

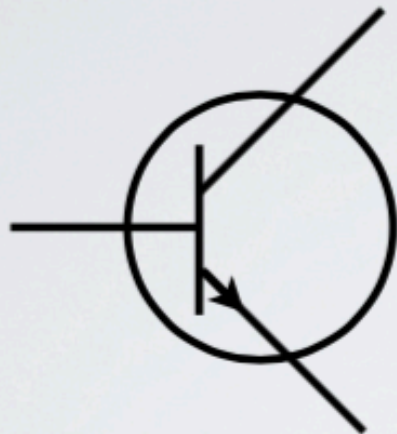


| A | B | A NOR B | A OR B |
|---|---|---------|--------|
| 0 | 0 | 1       | 0      |
| 0 | 1 | 0       | 1      |
| 1 | 0 | 0       | 1      |
| 1 | 1 | 0       | 1      |

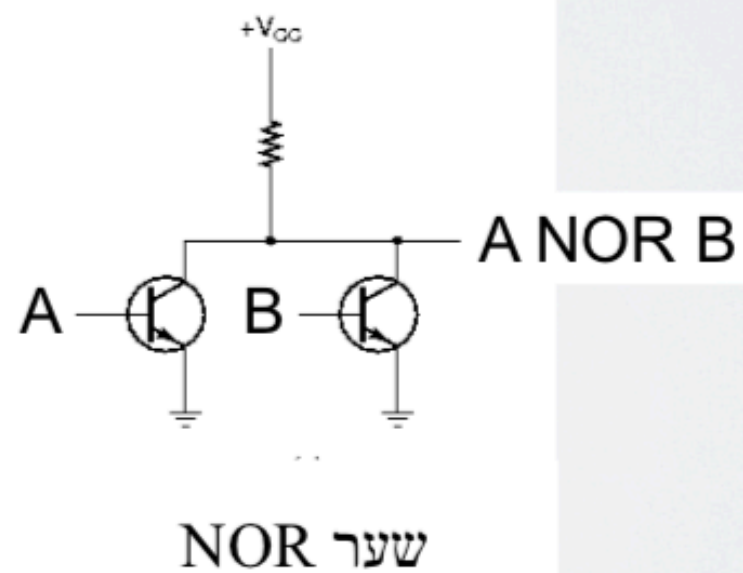


משפט: כל פונקציה בוליאנית  
מ-n ביטים ל-m ביטים ניתנת  
לחישוב ע"י הרכבה של שערי  
NOR.

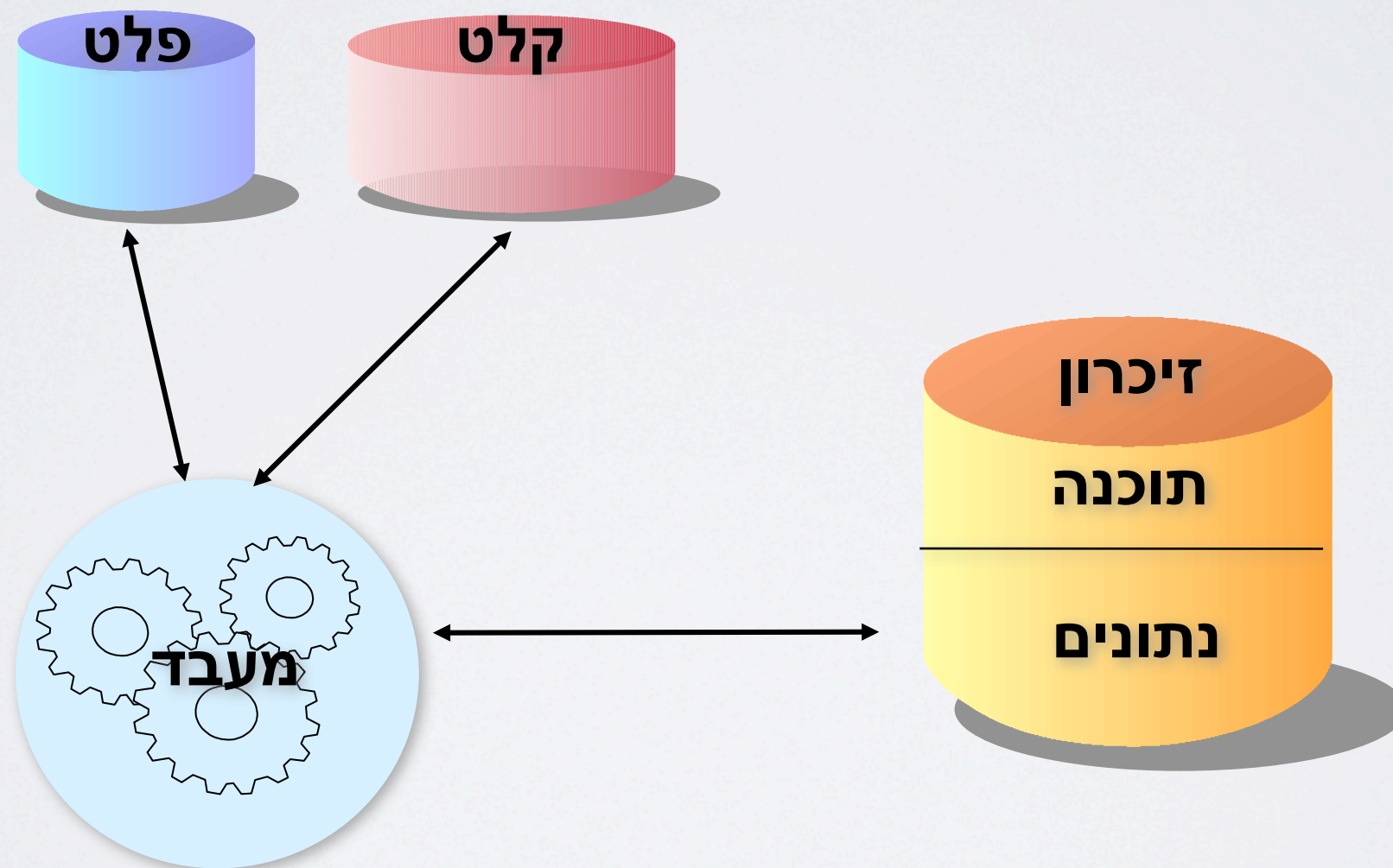
# מימוש על ידי טרנזיסטורים



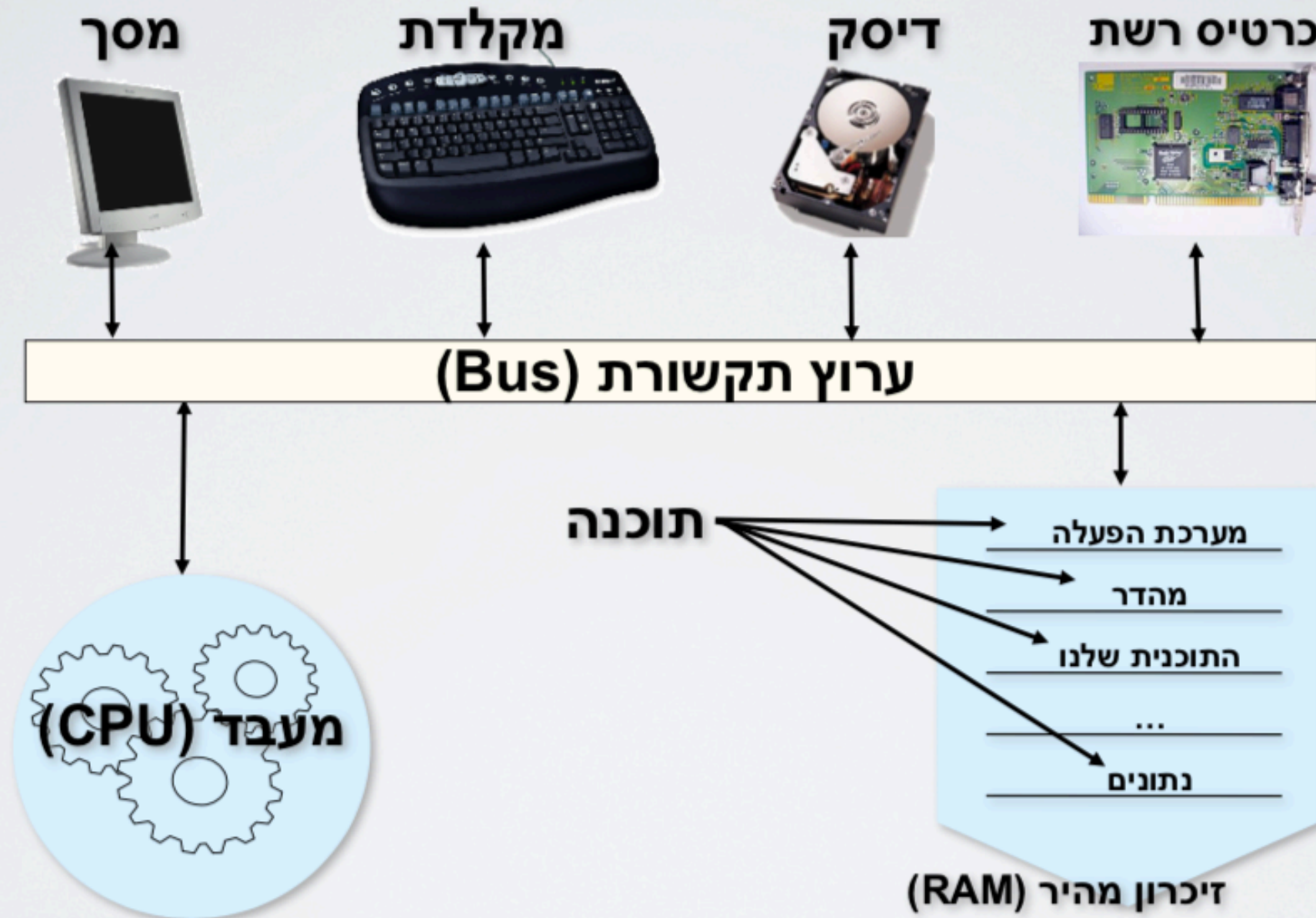
- 1 – יש זרם (TRUE)
- 0 – אין זרם (FALSE)



# תרשים מופשט של מחשב



# ארגון המחשב



# ייצוג בינארי

- הנתונים במחשב מיוצגים על ידי רצפים של ביטים  
הייצוג בדרך כלל לא מעניין אותנו

- אפשר לייצג כך סוגים שונים של נתונים:



מספרים שלמים:



מספרים רציונליים

$A = 00100001, B = 00100010, \dots$

אותיות

- # הסימנים האפשרי תלוי ב-# הביטים שמקצים להצגה:

2 ביטים נותנים 4 צירופים: 00, 01, 10, 11

8 ביטים נותנים 256 צירופים

32 ביטים (מילה) נותנים למעלה מ-4,000,000,000 צירופים

תוכנה

# אלגוריתמים

- אלגוריתם הוא מתכון לפיתרון של בעייה חישובית  
למשל: בדיקה אם הקלט הוא מספר זוגי

- האלגוריתם מתאר פיתרון בסדרת צעדים פשוטים  
קרא את הקלט  
בדוק אם הוא מספר שלם  
אם כן, חלק אותו ב-2, אחרת הודע על שגיאה  
אם המנה היא מספר שלם הדפס "זוגי", אחרת הדפס "אי-זוגי"

- מחשב חייב לקבל הוראות ברורות ומדוייקות

- תוכנית מחשב = אלגוריתם בפורמט ניתן להרצה במחשב

# שפת מכונה

- לכל מעבד יש שפת מכונה משלו
- שפת אסמבלי (assembly) מאפשרת לכתוב שפת מכונה בפורמט שקל לקריאה אנושית

שפת אסמבלי

load R7, value

jgt label3

add R7, bonus

שפת מכונה

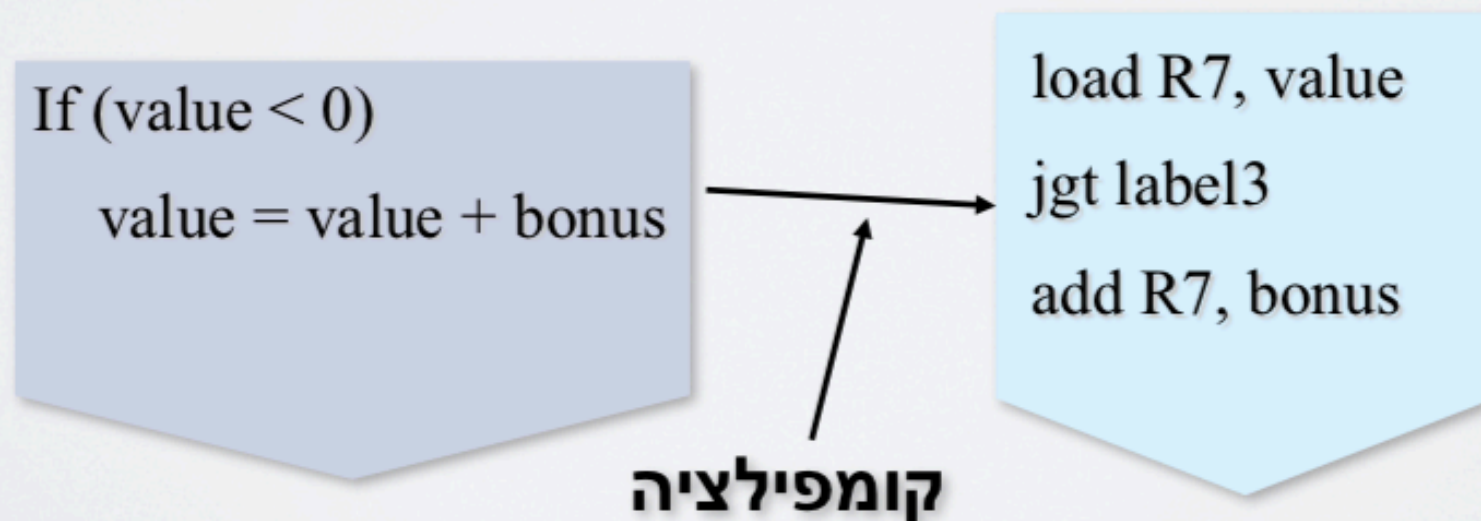
0100 0111 1101 0011 ...

0101 1011 0110 0111 ...

0110 0111 1010 1100 ...

# שפה עילית

- מתארת תוכנית מחשב ברמה גבוהה יותר משפת מכונה
- אפשר לתרגם תוכנית בשפה עילית לשפת מכונה (בעזרת מהדר - compiler) או להריצה ישירות בעזרת תוכנה מיוחדת (interpreter)



# מערכת הפעלה

מספקת שירותים תלויי-חומרה דרך ממשק לא תלוי-חומרה

מערכת קבצים

הדפסה

תקשורת

הפעלה והרצה של תוכניות

ניהול זיכרון

ממשק משתמש

ניהול תהליכים

שפת פייתון

# תכנות בפייתון

- נלמד פייתון בגרסה 3.X. היא שונה מהותית מגרסה 2.X.
- קוד פייתון מריצים דרך אינטרפרטר (interpreter).
- אפשר להקליד פקודות בודדות ולקבל חיווי מיידי של תוצאת הפעלתן, או לקודד תוכנית שלמה בקובץ ואז להריץ אותה.
- אני אדגים הרצה בסביבת עבודה מאוד פשוטה (IDLE).

# איפה מוצאים עזרה?

- הגישה האמפירית - הרבו בניסויים. המחשב לא יתפרק.
- דרך המרשתת - חפשו מידע דרך מנוע חיפוש.
- סביבות העבודה של פייתון מספקות גישה לתיעוד ולמדריכים.
- הגישה הלמדנית - ספרות, למשל:  
“Think Python, How to Think Like a Computer Scientist”

# תוכנית ראשונה בפייתון

Python 3.4.1: helloworld.py - /Users/yrabani/Dropbox/1 פייתון/תשעז/שיעור 1 - המחשב - Code snippets/hellowo...

```
print("Hello World!")
```

Ln: 2 Col: 0

# תוכנית ראשונה בפייתון

Python 3.4.1: helloworld.py - /Users/yrabani/Dropbox/1 פייתון/תשעז/שיעור 1/Code snippets/hellowo...

```
print("Hello World!")
```

Ln: 2 Col: 0

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Hello World!
>>>
```

Ln: 7 Col: 4

# הערה

- `print()` היא פונקצית מערכת של פייתון. היא יכולה לקבל סדרה של ערכי קלט, מופרדים בפסיקים, למשל:

```
print("Hello", "World!")
```

שמדפיסה

```
Hello World!
```

(הערכים בסדרה מופרדים בפסיקים)

# תוכנית בפייתון

- תוכנית בפייתון היא סדרה של פקודות (statements), כל אחת בשורה נפרדת, שמתבצעות לפי סדר:

Python 3.4.1: prints.py - /Users/yrabani/Dropbox/1 פייתון/תשעז/שיעור - Code snippets/prints.py

```
print("Once", "upon", "a", "time")  
print("...")  
print("and they all lived happily ever after.")
```

Ln: 3 Col: 47

# תוכנית בפייתון

- תוכנית בפייתון היא סדרה של פקודות (statements), כל אחת בשורה נפרדת, שמתבצעות לפי סדר:

Python 3.4.1: prints.py - /Users/yrabani/Dropbox/1 פייתון/תשעז/שיעור - Code snippets/prints.py

```
print("Once", "upon", "a", "time")
print("...")
print("and they all lived happily ever after.")
```

Ln: 3 Col: 47

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Once upon a time
...
and they all lived happily ever after.
>>>
```

# תיעוד

- קשה לעקוב אחרי קוד, ולכן נהוג להוסיף לקוד תיעוד (הערות):

```
Python 3.4.1: printswithcomments.py - /Users/yrabani/Dropbox/1 פייתון/תשעז/שיעור  
# this program demonstrates a sequence  
# of print statements  
print("Once", "upon", "a", "time") # thus begin fairy tales  
print("...")  
print("and they all lived happily ever after.") # and thus they end  
Ln: 5 Col: 67
```

- האינטרפרטר מתעלם מכל טקסט אחרי # ועד סוף השורה.
- אתם נדרשים לכתוב קוד מתועד היטב.
- בשל מגבלות המקום בשקפים, אנחנו נדגים לעיתים קוד נטול תיעוד.

# ביטויים אריתמטיים

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> 2+2
4
>>> 2*2
4
>>> 2**2
4
>>> 2**-2
0.25
>>> 3.7*5
18.5
>>> 3.7*5 + 9.2
27.7
>>> 3.7 * (5+9.2)
52.54
>>> 5/2
2.5
>>> 5//2
2
>>> 3*5/2
7.5
>>> 3//2*5
5
>>> 2.2 + 5/2
4.7
>>> (2.2+5) / 2
3.6
>>> 2.2 * 5**2
55.00000000000001
>>> 5%2
1
>>> print(9 / (4-2.5)**3)
2.6666666666666665
>>>
```

Ln: 36 Col: 4

# ביטויים אריתמטיים

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> 2+2
4
>>> 2*2
4
>>> 2**2
4
>>> 2**-2
0.25
>>> 3.7*5
18.5
>>> 3.7*5 + 9.2
27.7
>>> 3.7 * (5+9.2)
52.54
>>> 5/2
2.5
>>> 5//2
2
>>> 3*5/2
7.5
>>> 3//2*5
5
>>> 2.2 + 5/2
4.7
>>> (2.2+5) / 2
3.6
>>> 2.2 * 5**2
55.00000000000001
>>> 5%2
1
>>> print(9 / (4-2.5)**3)
2.6666666666666665
>>>
```

סימני הפעולות נקראים אופרטורים

# ביטויים אריתמטיים

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> 2+2
4
>>> 2*2
4
>>> 2**2
4
>>> 2**-2
0.25
>>> 3.7*5
18.5
>>> 3.7*5 + 9.2
27.7
>>> 3.7 * (5+9.2)
52.54
>>> 5/2
2.5
>>> 5//2
2
>>> 3*5/2
7.5
>>> 3//2*5
5
>>> 2.2 + 5/2
4.7
>>> (2.2+5) / 2
3.6
>>> 2.2 * 5**2
55.00000000000001
>>> 5%2
1
>>> print(9 / (4-2.5)**3)
2.6666666666666665
>>>
```

סימני הפעולות נקראים אופרטורים

הפעולות מתבצעות על פי כללי קדימות  
ואסוציאטיביות; אפשר להשתמש ב-()  
על מנת לכפות סדר רצוי

# טיפוסי אובייקטים

- לכל אובייקט (למשל, קבוע) בפייתון יש טיפוס (type):

| דוגמה            | טיפוס | הערך המוחזק            |
|------------------|-------|------------------------|
| -20              | int   | מספר שלם               |
| 5.3e-9 ,0.1      | float | מספר רציונלי           |
| “Hello”, ‘Hello’ | str   | מחרוזת (רצף של סימנים) |
| True, False      | bool  | ערך אמת                |

# משתנים והשמות

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> x = 15
>>> print(x)
15
>>> type(x)
<class 'int'>
>>> y = x + 2
>>> print(y)
17
>>> type(y)
<class 'int'>
>>> z = y * 2.2
>>> print(z)
37.400000000000006
>>> type(z)
<class 'float'>
>>> quote = 'to be or not to be'
>>> type(quote)
<class 'str'>
>>> x = quote
>>> print(x)
to be or not to be
>>> type(x)
<class 'str'>
>>>
```

# משתנים והשמות

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
```

```
>>> x = 15
>>> print(x)
15
>>> type(x)
<class 'int'>
>>> y = x + 2
>>> print(y)
17
>>> type(y)
<class 'int'>
>>> z = y * 2.2
>>> print(z)
37.400000000000006
>>> type(z)
<class 'float'>
>>> quote = 'to be or not to be'
>>> type(quote)
<class 'str'>
>>> x = quote
>>> print(x)
to be or not to be
>>> type(x)
<class 'str'>
>>>
```

שם משתנה חייב להתחיל באות או \_  
ויכול לכלול אותיות, ספרות, או \_

# משתנים והשמות

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
```

```
>>> x = 15
>>> print(x)
15
>>> type(x)
<class 'int'>
>>> y = x + 2
>>> print(y)
17
>>> type(y)
<class 'int'>
>>> z = y * 2.2
>>> print(z)
37.400000000000006
>>> type(z)
<class 'float'>
>>> quote = 'to be or not to be'
>>> type(quote)
<class 'str'>
>>> x = quote
>>> print(x)
to be or not to be
>>> type(x)
<class 'str'>
>>>
```

שם משתנה חייב להתחיל באות או \_  
ויכול לכלול אותיות, ספרות, או \_

כדאי לבחור למשתנים שמות משמעותיים  
שמעידים על מטרתם.

# המרת טיפוסים

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> type("I've failed over and over and over again in my life.")
<class 'str'>
>>> type("And that is why I succeed.")
<class 'str'>
>>> type(3)
<class 'int'>
>>> type(3.0)
<class 'float'>
>>> type(12.9)
<class 'float'>
>>>
>>>
>>> int(12.9)
12
>>> str(12.9)
'12.9'
>>> float(3)
3.0
>>>
```

# פיתרון משוואה ריבועית

Python 3.4.1: quadeq.py - /Users/ywabani/Dropbox/1/תשעז/שיעור פיתון/Code snippets/qu...

```
import math
```

```
# print the two solutions to the equation  $x^2 - 4x + 3 = 0$ 
```

$$a = 1.0$$
$$b = -4.0$$
$$c = 3.0$$

```
radical = math.sqrt(b**2 - 4*a*c) # 4**2 - 4*3 > 0!
```

$$\text{sol1} = (-b - \text{radical}) / (2*a)$$
$$\text{sol2} = (-b + \text{radical}) / (2*a)$$

```
print("the two solutions are:", sol1, sol2)
```

Ln: 11 Col: 30

# תוכנית עם באג (טעות)

```
width = input("Enter rectangle's width: ")
height = input("Enter rectangle's height: ")

area = width * height
print("The area is: ", area)
```

# תוכנית עם באג (טעות)

```
width = input("Enter rectangle's width: ")
height = input("Enter rectangle's height: ")

area = width * height
print("The area is: ", area)
```

```
Enter rectangle's width: 4
Enter rectangle's height: 3
Traceback (most recent call last):
  File "C:/Users/avivz/Desktop/tmp/expressions.py", line 4, in
<module>
    area = width * height
TypeError: can't multiply sequence by non-int of type 'str'
```

# תוכנית עם באג (טעות)

```
width = input("Enter rectangle's width: ")
height = input("Enter rectangle's height: ")

area = width * height
print("The area is: ", area)
```

```
Enter rectangle's width: 4
Enter rectangle's height: 3
Traceback (most recent call last):
  File "C:/Users/avivz/Desktop/tmp/expressions.py", line 4, in
<module>
    area = width * height
TypeError: can't multiply sequence by non-int of type 'str'
```

```
width = int(input("Enter rectangle's width: "))
height = int(input("Enter rectangle's height: "))

area = width * height
print("The area is: ", area)
```

# באג ב-Harvard mark II, 1947

9/9

0800 Antan started  
 1000 " stopped - antan ✓  
 1300 (032) MP - MC ~~1.982147000~~  
 (033) PRO 2 2.130476415  
 connect 2.130676415

Relays 6-2 in 033 failed special speed test  
 in relay " 11.00 test.

Relay  
 2145  
 Relay 3376

1100 Started Cosine Tape (Sine check)  
 1525 Started Multi-Adder Test.

1545



Relay #70 Panel F  
 (moth) in relay.

First actual case of bug being found.  
 1630 Antan started.  
 1700 closed down.